基于 Clang/LLVM 的语言处理流程、SysY 实现与 AscendNPU IR 实验

甘文杰（学号：2412700）

向宇航（学号：2413318）

2026 年 10 月 7 日

摘要

本实验以一段包含函数、循环、分支和输入输出的 SysY 程序观察 Clang/LLVM 的处理过程，并独立编写 LLVM IR 与 RV64 汇编，用六组输入核对运行结果。实验中检查了预处理文本、LLVM IR、汇编和 ELF 目标文件，也观察了链接后程序如何解析运行库函数。优化部分记录本例在 x86-64 与 RV64 目标上的不同变化。进阶部分从源码构建 AscendNPU-IR 的 `bishengir-opt`，按上游测试命令运行 PlanMemory 和 InjectSync，并用上游 FileCheck 断言验证输出。两组 Before/After 均收录于报告附录。

关键词：编译器；Clang；LLVM IR；RISC-V；ABI；MLIR；lowering

分工

甘文杰：负责基础内容撰写，并对 Ascend 部分进行 Review。

向宇航：负责环境配置、资料查找和 Ascend 部分撰写，并对基础部分进行 Review。

双方协作完成报告的整合与定稿。

目录

1 引言 **4**

2 实验设计与工具环境 **4**

2.1 示例程序与语言特性 4

2.2 基准结果与验证方法 4

2.3 基础实验工具链 5

3 Clang 语言处理流程与阶段产物 **5**

3.1 处理流程总览 5

3.2 预处理：源文本与行标记 5

3.3 编译：前端、中端与后端 6

3.4 LLVM IR 的生成命令与产物 6

3.5 汇编：机器码、符号与重定位 6

3.6 链接：运行库解析与可执行文件 7

4 等价程序的手写实现与运行验证 **8**

4.1 LLVM IR 的控制流与源语句对应 8

4.2 手写 LLVM IR 与生成结果对比 8

4.3 RISC-V 参考汇编与调用约定 10

4.4 栈帧布局 11

4.5 手写 RISC-V 汇编 11

4.6 SysY 运行库链接与 QEMU 执行 12

4.7 三种实现的运行结果比较 12

5 优化探索：-O0 与 -O2 **13**

5.1 对照方法与总体变化 13

5.2 从内存槽到 SSA 13

5.3 奇偶判断与分支化简 13

5.4 x86-64 与 RV64 的目标差异 14

5.5 优化观察的结论边界 14

6 进阶探索：MLIR 与 AscendNPU IR **14**

6.1 多级 IR 与渐进式 lowering 14

6.2 VecAdd 的概念路线与 HIVM 表示 14

6.3 构建环境、链接补丁与验证方法 15

6.4 PlanMemory：UB 地址规划实测 15

6.5 InjectSync：流水线依赖实测 16

6.6 探索结果与实测边界 16

7 结论 17

A 实验复现说明与材料索引 18

A.1 环境、文件组织与外部依赖 18

A.2 各阶段复现脚本总表 18

A.3 正文结论与附录材料对应表 19

B 基础实验输入与完整手写实现 20

B.1 SysY 源程序与 Clang 输入版本 20

B.2 宿主机输入输出运行库 21

B.3 完整手写 LLVM IR 21

B.4 完整手写 RISC-V 汇编 23

C 编译流程与运行验证记录 25

C.1 预处理、IR、汇编和目标文件生成命令 25

C.2 ELF 文件头、符号、重定位与链接日志摘录 25

C.3 自动生成的 LLVM IR 与 RV64 参考汇编 28

C.4 手写实现的编译、链接与执行命令 32

C.5 固定测试方法、逐组输出与一致性结果 32

D 优化对照过程材料 34

D.1 O0/O2 生成命令与指令统计口径 34

D.2 LLVM IR 关键变化及原始片段 34

D.3 x86-64 与 RV64 汇编对照片段 36

E AscendNPU IR 实验过程材料 39

E.1 官方 VecAdd HIVM 完整输入 39

E.2 构建版本、配置、故障处理与完整链接补丁 39

E.3 三条上游 RUN/FileCheck 命令与通过记录 41

E.4 PlanMemory 单用例完整 Before/After 42

E.5 InjectSync 单用例完整 Before/After 43

1 引言

本次作业需要将中间产物分别保存下来，再检查每一步具体做了什么。LLVM IR 可用于追踪源程序中的变量和控制流；RISC-V 汇编则展示参数、返回值和局部变量如何使用寄存器与栈帧。

将求和程序作为示例：计数器 i 从 1 增加到输入 n，偶数项按 $2i$ 累加，奇数项按 i 累加。程序包含局部变量、算术、比较、条件分支、循环、函数调用和运行库输入输出，覆盖作业要求观察的主要结构。后续实现使用同一组输入。

先观察 Clang 生成的中间文件和目标文件，再对照 LLVM IR 的基本块结构，完成手写 IR 与汇编，最后比较优化结果并记录 AscendNPU IR 的 Pass 实验。正文保留理解这些结果所需的代码和数据；完整输入、实现以及必要命令与过程证据收录于附录，读者仅凭本报告即可查阅。附录材料索引见附录 A。

2 实验设计与工具环境

2.1 示例程序与语言特性

本次实验使用同一个程序。SysY 源程序如下，完整输入版本见附录 B.1。

```
int calc(int n) {
    int i = 1;
    int sum = 0;
    while (i <= n) {
        if (i % 2 == 0) { sum = sum + i * 2; }
        else            { sum = sum + i;     }
        i = i + 1;
    }
    return sum;
}

int main() {
    int n;      n = getint();
    int result; result = calc(n);
    putint(result);
    return 0;
}
```

程序从 $i = 1$ 开始循环，直到 $i > n$ 后返回累加值。为让 Clang 直接处理，附录 B.1 中的 C 输入版本保留相同算法，只补充 `getint`/`putint` 的函数声明。

2.2 基准结果与验证方法

固定输入为 $1, 2, 3, 4, 5, 10$，预期结果分别为 $1, 5, 8, 16, 21, 85$。这组结果由程序语义确定，再用于比较三个实现。

验证时以程序语义确定的结果为基准，比较宿主机参考实现、手写 LLVM IR 和手写 RISC-V 的标准输出；课程 runtime 的 stderr 计时信息不计入结果。六组固定输入覆盖了不同循环长度与奇偶项组合，未穷举全部输入。完整实现、测试方法与原始输出见附录 B 和附录 C.5。

2.3 基础实验工具链

表 1: 本实验使用的工具链

工具	版本	本实验用途
Clang / LLVM	14.0.0	预处理、LLVM IR、x86-64 汇编、目标文件
GCC / GNU ld	11.4.0 / 系统 GNU ld	系统编译与链接工具
RISC-V unknown-elf GCC	15.1.0	汇编、链接手写 RV64 汇编与课程 runtime
QEMU	7.0.0	用户态运行 RISC-V 可执行文件

3 Clang 语言处理流程与阶段产物

3.1 处理流程总览

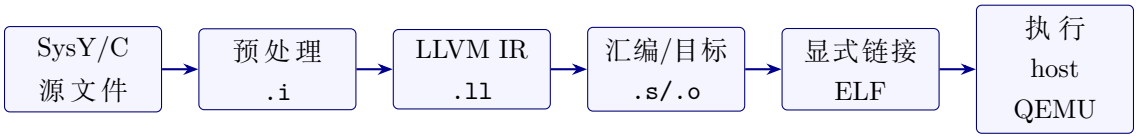


图 1: 从源程序到运行文件的实验流程

3.2 预处理：源文本与行标记

首先运行 `clang -E source/example_clang.c -o generated/example.i` 生成预处理结果。预处理器展开宏、插入头文件、处理条件编译，并在输出中保留源位置标记；未检查程序的类型或控制流。本例没有宏和头文件，`generated/example.i` 的程序语句与输入相同，文件中多出 Clang 写入的行标记。

```
# 1 "source/example_clang.c"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 361 "<built-in>" 3
...
# 1 "source/example_clang.c" 2
int getint(void);
void putint(int x);
int calc(int n) {
    int i = 1;
    ...
```

行标记记录源文件与行号，供后续工具定位代码来源；<built-in> 表示编译器提供的内建内容。

3.3 编译：前端、中端与后端

对同一份源文件，clang -cc1 依次完成以下工作；本实验通过 example.ll 中的结果反推各阶段产物。

表 2: 编译器内部阶段及其在本程序中的表现

阶段	职责	本程序的观察
词法分析	把字符流切成记号	int、calc、i、<=、2 等记号
语法分析	按文法构造抽象语法树	得到 while、if/else 的嵌套结构
语义分析	类型检查、建立符号表、插入隐式转换	判定 i <= n 为 i32 比较，calc(n) 类型匹配
IR 生成	遍历 AST 生成 LLVM IR	局部变量变为 alloca，语句变为基本块与 br
中端优化	在 IR 上做与目标无关的变换	本例 -O0 保留栈槽；-O2 中出现 SSA 与 phi（第 5 节）
后端	指令选择、寄存器分配、指令调度	生成 x86-64 或 RV64 汇编

词法、语法和语义分析通常归入前端；中端处理 IR，后端再面向目标架构。表中的观察来自这一个程序和 Clang 生成的文件。

3.4 LLVM IR 的生成命令与产物

以下命令输出 LLVM IR：

```
clang -S -emit-llvm -O0 source/example_clang.c -o generated/example.ll
```

命令生成 generated/example.ll。在这份 -O0 IR 中，函数和控制流已显式写出；局部变量通过 alloca 取得栈槽，再由 load/store 读写。第 4.1 节进一步对照了这些指令与源程序的关系。

3.5 汇编：机器码、符号与重定位

clang -S -O0 生成宿主机汇编 generated/example_x86.s；clang -c -O0 继续调用集成汇编器，将汇编编码为机器码，并输出 ELF64 可重定位文件 generated/example.o。汇编器同时记录节区、符号和重定位信息；外部符号的最终地址由链接器确定。对 example.o 的检查结果如下。

表 3: example.o 的 ELF 特征（file 与 readelf 实际输出）

检查项	实际结果
文件类型	ELF 64-bit LSB relocatable, x86-64, not stripped，即 ET_REL
主要节区	.text、.rela.text、.eh_frame、.symtab、.strtab
已定义符号	calc、main，类型为 FUNC
未定义符号	getint、putint，均为 UND
重定位项	.rela.text 中有 R_X86_64_PLT32 项，涉及函数调用

readelf -h 给出的 ELF 文件头也印证了这一点：

```
Class:                             ELF64
Type:                              REL (Relocatable file)
Machine:                           Advanced Micro Devices X86-64
Entry point address:               0x0
Start of program headers:          0 (bytes into file)
```

Type 为 REL，且该文件没有可执行文件所需的入口布局，因此不能直接作为进程运行。符号表中，calc/main 是本文件定义的函数，getint/putint 则仍需由其他目标文件提供。重定位项记录了调用位置需要链接器补上的地址信息；其中既有运行库函数，也有对 calc 的调用。由此可以区分“汇编已经编码”和“程序已经可以运行”这两个状态。

3.6 链接：运行库解析与可执行文件

本实验先分别生成 example.o 与 host_runtime.o，再用 Clang 驱动显式链接。日志显示驱动调用系统链接器，并加入启动文件和 C 库。链接器把 example.o 中对 getint/putint 的引用与 runtime 中的定义对应起来，再处理重定位并生成可执行文件 generated/reference_linked。它的六组输出与基准一致。单独的 example.o 仍是可重定位文件，也保留未定义的外部符号，因此不能独立运行。

从这些文件可以看到，预处理结果仍是源文本，.ll 显示控制流和运算，.s 面向具体处理器，而 .o 已将指令编码并记录尚待链接器处理的引用。各阶段产物与相应职责由此对应起来。

各阶段完整命令、目标文件检查与链接日志摘录见附录 C.1 和附录 C.2；自动生成的基础 IR 与 RV64 参考汇编见附录 C.3。

4 等价程序的手写实现与运行验证

4.1 LLVM IR 的控制流与源语句对应

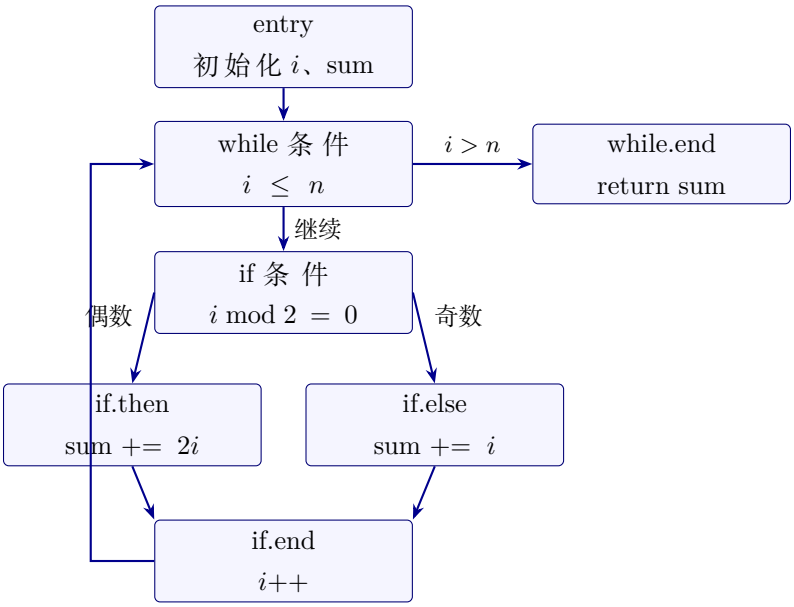


图 2: `calc` 的基本块与循环回边

Clang -O0 生成的 `calc` 使用数字标签表示基本块。入口初始化 `i` 和 `sum` 后，条件块比较 i 与 n；循环体再判断奇偶并分到两个累加分支。分支合流后递增 `i` 并跳回条件块，形成循环回边。离开循环时，程序读取 `sum` 并返回。由此可见，源语言的 `while` 和 `if` 没有对应的专用指令，而是由基本块、比较和 `br` 组合表示。

表 4: 源程序结构与 LLVM 指令对应

源程序结构	LLVM 指令	作用
局部变量声明/赋值	alloca、store	为 n/i/sum 分配栈槽并写初值或新值
读取变量	load	取 i、n、sum 的当前值
加法、乘法、取模	add、mul、srem	计算计数器、累加项与奇偶性
比较	icmp sle、icmp eq	判断循环范围和余数是否为零
if/while	br、基本块	根据条件转移，合并分支并形成循环回边
函数关系	define、call、ret	定义 calc/main，调用 getint/calc/putint 并返回

4.2 手写 LLVM IR 与生成结果对比

基于 CFG 独立编写了附录 B.3 中的 LLVM 模块，没有复制 Clang 的 IR。手写版使用 `while.cond` 等语义化块名；形参 `n` 直接作为 SSA 值使用，只有 `i` 和 `sum` 保存在栈槽中。下面列出循环条件和回边，完整模块见附录 B.3。

```
define i32 @calc(i32 %n) {
entry:
```

```
  %i.addr = alloca i32, align 4
  %sum.addr = alloca i32, align 4
  store i32 1, i32* %i.addr, align 4      ; i = 1
  store i32 0, i32* %sum.addr, align 4    ; sum = 0
  br label %while.cond

while.cond:
  %i.cond = load i32, i32* %i.addr, align 4
  %continue = icmp sle i32 %i.cond, %n    ; i <= n
  br i1 %continue, label %if.cond, label %while.end

if.cond:
  %i.parity = load i32, i32* %i.addr, align 4
  %remainder = srem i32 %i.parity, 2      ; i % 2
  %is.even = icmp eq i32 %remainder, 0
  br i1 %is.even, label %if.then, label %if.else

  ; 两个累加分支更新 sum 后进入 if.end

if.end:
  %i.current = load i32, i32* %i.addr, align 4
  %i.next = add nsw i32 %i.current, 1     ; i = i + 1
  store i32 %i.next, i32* %i.addr, align 4
  br label %while.cond                    ; while 的回边

while.end:
  %sum.result = load i32, i32* %sum.addr, align 4
  ret i32 %sum.result
}
```

`while.cond` 用条件分支决定继续循环还是退出；两个累加分支在 `if.end` 汇合，随后无条件跳回循环条件。Clang 接受完整的手写 IR，并将它与附录 B.2 的宿主机运行库链接。六组固定输入均通过，逐组原始结果见附录 C.5。

表 5: 自动生成 IR 与手写 IR 的组织方式对比

对比项	自动生成 IR（附录 C.3）	手写 IR（附录 B.3）
基本块命名	数字标签 %5、%9、%13	语义名 while.cond、if.then、if.end
局部变量	三个 alloca，含形参 n 的栈槽	只给 i、sum 两个 alloca，n 直接用 SSA 形参
分支合流	%13、%18 汇入 %22	if.then、if.else 汇入 if.end
元数据	带 attributes、!llvm.loop、!llvm.module.flags	不产生任何元数据
目标三元组	x86_64-pc-linux-gnu 与完整 datalayout	仅声明 target triple
指令集合	同一组核心指令	逐条对应源语句，无新增或缺失的操作

两份 IR 的核心运算和控制流相同，但块名、元数据以及局部变量的存放方式不同（表 5）。手写过程先按源程序整理控制流，再逐个填写运算和跳转，便于检查语义。

4.3 RISC-V 参考汇编与调用约定

使用 Clang 14 交叉生成 RV64 参考汇编（`--target=riscv64-unknown-elf -march=rv64gc -mabi=lp64d`），并将后端输出与手写实现对照。`calc` 的序言和奇偶判断片段如下：

```
calc:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)
    addi s0, sp, 32
    sw   a0, -20(s0)
    ...
.LBB0_1:
    lw   a1, -24(s0)      # i
    lw   a0, -20(s0)      # n
    blt  a0, a1, .LBB0_6
    ...
.LBB0_2:
    lwu  a0, -24(s0)      # i % 2 的替代实现
    srli a1, a0, 31
    add  a1, a1, a0
    andi a1, a1, -2
    subw a0, a0, a1
    bne  a0, zero, .LBB0_4
```

与手写版本相比，这次 Clang 输出的 `calc` 与手写版都使用 32 字节栈帧，`ra`、`s0` 和局部变量的位置也相同。两版对奇偶性的处理不同：Clang 把已知除数为 2 的余数计算化成一组移位和算术指令；手写版直接用 `remw`。`rv64gc` 包含整数乘除扩展，因此 `remw` 是可用指令，Clang 选

择算术展开并非因为架构没有取余操作。这个例子中 i 从正数 1 开始递增，所以该变换与程序使用范围相符。两版的 main 栈帧也不同：Clang -O0 使用 48 字节，手写版使用 16 字节，反映出未优化代码和人工组织值的方式不同。

表 6: 本实验用到的 RISC-V 寄存器与调用约定

寄存器/规则	本实验用法
a0	第一个整型参数与整型返回值；getint、calc、putint 之间传值
t0-t2	calc 内的 caller-saved 临时值；本实验只用这三个
s0	calc 的 frame pointer；属被调用者保存，成对保存/恢复
sp	栈指针；calc/main 分别保留 32/16 字节且保持 16 字节对齐
ra	返回地址；calc 与 main 跨越函数调用保存/恢复

4.4 栈帧布局

RV64 调用约定要求 sp 16 字节对齐。calc 的栈帧共 32 字节：顶部保存返回地址与旧的 s0，中部放三个局部变量。堆栈从高地址向低地址增长，因此“入栈”即减小 sp。

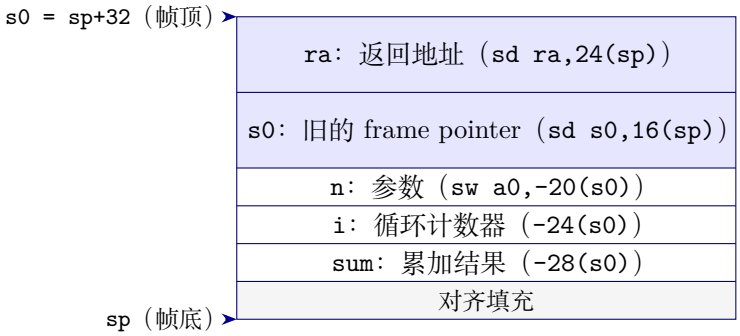


图 3: calc 的 32 字节栈帧（addi sp,sp,-32）

4.5 手写 RISC-V 汇编

附录 B.4 中的手写汇编独立遵守同一约定：序言建立栈帧并保存 ra/s0，循环条件用 blt 在 i > n 时退出，奇偶判断用 remw 加 beqz，偶数项用 slliw 左移一位代替乘 2，累加用 addw，循环尾递增 i 后无条件跳回条件块。控制流标签与第 4.1 节的 LLVM 基本块一一对应。

```
calc:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)
    addi s0, sp, 32
    sw   a0, -20(s0)       # 保存参数 n
    ...
.Lwhile_cond:
    lw   t0, -24(s0)       # i
    lw   t1, -20(s0)       # n
```

```
    blt  t1, t0, .Lwhile_end
.Lif_cond:
    li   t1, 2
    remw t2, t0, t1
    beqz t2, .Lif_then
    ...
.Lwhile_end:
    lw   a0, -28(s0)       # 返回值
    ld   s0, 16(sp)
    ld   ra, 24(sp)
    addi sp, sp, 32
    ret
```

完整实现见附录 B.4。其中 `main` 用 16 字节栈帧保存 `ra`，依次调用 `getint`、`calc`、`putint`；整数参数和返回值都经 `a0` 传递。

4.6 SysY 运行库链接与 QEMU 执行

使用 `riscv64-unknown-elf-gcc` 汇编手写程序，并静态链接课程提供的 `libsysy_riscv.a`。链接配置为静态模式，采用 `medany` 代码模型，文本段地址设为 `0x90000000`：

```
-static -mcmodel=medany -Wl,--no-relax,-Ttext=0x90000000
```

产物 `generated/manual_riscv` 是 RISC-V ELF 可执行文件，入口地址为 `0x90000044`，由 `qemu-riscv64` 用户态模拟执行。runtime 的析构函数会向 stderr 打印 `TOTAL: 0H-0M-0S-0us` 计时摘要。

4.7 三种实现的运行结果比较

表 7: 六组输入在三种实现上的输出比较

输入 n	Reference	手写 LLVM	手写 RISC-V/QEMU	结果
1	1	1	1	PASS
2	5	5	5	PASS
3	8	8	8	PASS
4	16	16	16	PASS
5	21	21	21	PASS
10	85	85	85	PASS

逐组原始输出和统一比对记录见附录 C.5。这组测试确认了固定输入上的结果一致性，不构成对任意输入的形式化等价证明。

5 优化探索：-O0 与 -O2

5.1 对照方法与总体变化

以同一份 C 文件生成两种优化级别的 LLVM IR、x86-64 汇编和 RV64 汇编，再比较实际 diff。下面的观察只针对这个程序和这两种目标。

表 8: LLVM IR 指令数量与结构变化

指标	-O0	-O2
`alloca`	6	0
`load`	11	0
`store`	9	0
`phi`	0	32
局部变量表示	栈槽 + 内存读写	SSA 临时值 + 循环 `phi`
奇偶判断	`srem` + `icmp eq` + `br`	`and`/`xor`/`shl` 位运算
`main` 对 `calc` 的调用	存在 `call @calc`	已内联；`@calc` 定义因外部可见而保留

5.2 从内存槽到 SSA

在这份 `-O0` IR 中，局部变量通过栈槽和 `load`/`store` 访问；`-O2` 结果则把这些变量保存在 SSA 值中，并用 `phi` 传递循环迭代间的值。统计结果为 `alloca`/`load`/`store` 从 6/11/9 降到 0/0/0，`phi` 从 0 增至 32。主要变化是同一状态不再通过内存读写表达。

5.3 奇偶判断与分支化简

源程序用 `i % 2 == 0` 选择分支。`-O2` 把奇偶判断从取模与分支改写为位运算。RV64 的 `calc` 中，奇偶位决定左移 0 位还是 1 位，因而两条累加路径可以合成一条：

```
calc:
    li   a1, 0
    blez a0, .LBB0_3
    li   a2, 1
.LBB0_2:
    mv   a3, a2
    not  a2, a2
    andi a2, a2, 1
    sllw a2, a3, a2        # i << (i 为偶数 ? 1 : 0)
    addw a1, a1, a2        # sum += 该项
    addiw a2, a3, 1
    blt  a3, a0, .LBB0_2
.LBB0_3:
    mv   a0, a1
    ret
```

not/andi 生成移位量，sllw 再按该值左移：偶数项乘 2，奇数项保持原值。这样，输出汇编不再保留原来的 if/else。优化后的 calc 也不再需要栈帧，因为它不调用其他函数且只使用临时寄存器。

5.4 x86-64 与 RV64 的目标差异

同一份较高层的优化结果，最终仍要由目标后端决定怎样使用本机指令。

x86-64 的结果中可以看到四个 i32 组成的向量运算、XMM 寄存器和 paddd；循环末尾还保留处理不足四个元素的路径。RV64 目标为 rv64gc，不含 RVV 扩展，所以输出仍是标量循环，没有 RVV 向量指令。两种目标都把 calc 内联进 main，但向量化结果不同。

5.5 优化观察的结论边界

两种目标生成的汇编长度变化方向不同：x86-64 因向量循环和内联而增加，RV64 因控制流与栈访问减少而变短。汇编文本行数不能替代运行时间、代码体积或能耗测量；本实验没有测量这些指标，因此不据此推断性能提升。

生成命令、统计口径及带上下文的原始对照片段见附录 D。上述计数针对完整宿主机 IR 模块，不能与仅统计 calc 的结果混用。

6 进阶探索：MLIR 与 AscendNPU IR

6.1 多级 IR 与渐进式 lowering

前文的 LLVM IR 已显式表示变量读写和分支；面向 NPU 时，还需要表达张量形状、存储位置、数据搬运和硬件流水线。MLIR 允许编译器用不同 Dialect 保留这些层次的信息，再逐步 lowering。较高层的张量与算子语义有利于融合和 tiling；HIVM 则能表达 GM、UB 以及对应的数据搬运和向量运算。图 4 是依据课程要求与官方资料整理的概念路线。[1, 2, 4]

6.2 VecAdd 的概念路线与 HIVM 表示

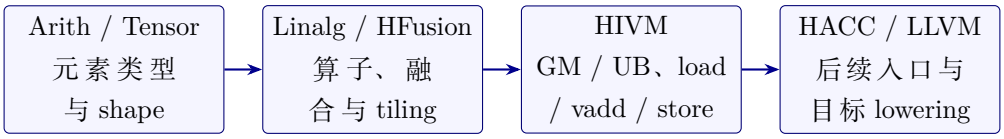


图 4: VecAdd 的概念 lowering 路线

这条概念路线中，高层表示主要描述“两个向量逐元素相加”；到了 HIVM，内存空间和搬运步骤已显式出现。官方 VecAdd 样例使用 16 个 i16 元素：A、B、C 位于 GM，输入通过 hivm.hir.load 搬入 UB，在 UB 上执行 hivm.hir.vadd，再由 hivm.hir.store 写回 GM。下列代码是对上游文件的结构摘录。[4, 5]

```
func.func @add(%a: memref<16xi16, #hivm.address_space<gm>>,
               %b: memref<16xi16, #hivm.address_space<gm>>,
```

```
                %c: memref<16xi16, #hivm.address_space<gm>>) {
  %ua = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.load ins(%a : ...) outs(%ua : ...) // GM -> UB
  %ub = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.load ins(%b : ...) outs(%ub : ...) // GM -> UB
  %uc = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.vadd ins(%ua, %ub : ...) outs(%uc : ...)
  hivm.hir.store ins(%uc : ...) outs(%c : ...) // UB -> GM
  return
}
```

代码摘录保留了本例的关键操作。`memref.alloc` 标出 UB 临时区，`load/store` 标出 GM 与 UB 之间的数据移动；算术本身由 `vadd` 表达。硬件相关信息进入 IR 后，后续 Pass 才能针对这些存储和执行约束继续处理。

6.3 构建环境、链接补丁与验证方法

构建环境使用用户级虚拟环境中的 CMake 4.4.4 和 Ninja 1.13.2。AscendNPU-IR 与 LLVM 子项目分别固定在提交 `2ac1b376` 和 `06913395`；构建出的工具报告版本为：

```
bishengir-opt 2.0.0 (gitcode.com/Ascend/ascendnpu-ir 2ac1b376da1d)
llvm 19.1.7 06913395ef8a   Release build
```

构建工具时遇到上游静态库之间的循环依赖。本地补丁使用 CMake 的 `LINK_GROUP:RESCAN` 让链接器重新扫描相关静态库；补丁只改变 `bishengir-opt` 的链接参数，没有修改 Pass 实现。详细构建过程和完整补丁见附录 E.2。随后三条上游 RUN 命令全部通过 FileCheck，包括 PlanMemory 的两种策略和 InjectSync。

6.4 PlanMemory：UB 地址规划实测

PlanMemory 根据临时 buffer 的生命周期和目标硬件约束安排片上地址；在这个用例中，它把 `memref.alloc` 改写为带偏移的 `hivm.hir.pointer_cast`。实验采用上游 `plan-memory.mlir` 中的 `test_mem_allocate_basic`：其中四个 UB buffer 都是 $16 \times 16 \times 16$ 的 `f16` 张量，每个占 8192 字节。测试默认策略命令为：

```
// RUN: bishengir-opt %s -hacc-append-device-spec=target=Ascend910B1 \
//        -hivm-plan-memory -split-input-file -verify-diagnostics | FileCheck %s
// CHECK-LABEL: func.func @test_mem_allocate_basic
// CHECK-NOT: memref.alloc()
// CHECK: arith.constant 0 : i64
// CHECK: hivm.hir.pointer_cast
```

`-hacc-append-device-spec` 先把 Ascend910B1 的目标规格写入模块，随后运行 PlanMemory。测试文件还包含一条使用 `largest-first` 策略的 `RUN`。两条命令均按上游定义执行，并交由 FileCheck 断言检查；结果均通过。

对默认策略用例保存的 Before/After 显示：

表 9: PlanMemory 在该用例上的实测变化

指令计数	Before	After
`memref.alloc()`	4	0
`hivm.hir.pointer_cast`	0	4
`hivm.hir.load/vadd/store`	5	5

输出中四个分配的形式均为 `pointer_cast(%c0_i64)`。它们的偏移均为 0；`load`、三次 `vadd` 和 `store` 保持不变。模块里还出现 UB 容量、对齐等目标参数。这些参数由命令中的 `-hacc-append-device-spec` 加入，不是 PlanMemory 自己生成的。该用例的 offset-0 结果只描述这组输入和策略；更复杂的生命周期和别名关系可能得到不同布局。完整 Before/After 见附录 E.4。[7, 8, 9]

6.5 InjectSync：流水线依赖实测

第二个上游用例 `test_mem_injcet_sync_basic` 只有 `load`、`vadd` 和 `store` 三个操作，输入中没有同步指令。运行测试中的 `-hivm-inject-sync -hivm-lower-multi-buffer-counter` 后，IR 增加了以下依赖关系：

```
hivm.hir.load ins(%arg0 : ...<gm>) outs(%0 : ...<ub>)
hivm.hir.set_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
hivm.hir.wait_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
hivm.hir.vadd ins(%0, %2 : ...) outs(%0 : ...)
hivm.hir.set_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
hivm.hir.wait_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
hivm.hir.store ins(%0 : ...<ub>) outs(%arg1 : ...<gm>)
hivm.hir.pipe_barrier[<PIPE_ALL>]
```

MTE2 与 MTE3 表示数据搬运流水，V 表示向量计算流水。Pass 在 load 后加入 MTE2 到 V 的 flag/wait，在 store 前加入 V 到 MTE3 的 flag/wait，并在函数末尾加入一条 `PIPE_ALL` barrier。统计结果是 2 条 `set_flag`、2 条 `wait_flag` 和 1 条 `pipe_barrier`；Before 文件中的真实指令数为 0。这组结果说明，IR 中的操作顺序并不等同于不同硬件流水线之间的完成依赖；Pass 将需要等待的依赖显式写出。完整 Before/After 见附录 E.5。[10, 8]

6.6 探索结果与实测边界

官方 VecAdd、PlanMemory 和 InjectSync 使用不同输入。两个 Pass 的单用例 Before/After 用于解释局部变换；三个上游 RUN 的 FileCheck 结果用于验证完整上游测试文件。不能将它们视为同一个 VecAdd 从高层到机器码的完整流水线转储。Arith/Tensor 与 HFusion 的路线属于资料整理，本次实测从 HIVM 开始，未安装 CANN、配置驱动、生成并运行 kernel 目标文件或上板。完整材料见附录 E。

7 结论

本次实验将源程序、LLVM IR、汇编、目标文件和可执行程序联系起来。检查 `example.o` 可见，它仍是 `ET_REL` 文件，并保留对 `getint/putint` 的引用；显式链接之后，这些引用才由 runtime 提供定义。由此可以区分汇编器与链接器的职责。

LLVM IR 中，循环条件、奇偶分支和回边由基本块与 `br` 指令表示，并据此完成了手写 IR。RV64 汇编展示了同一控制流在目标架构上的实现：参数和返回值使用 `a0`，`calc` 通过栈帧保存局部变量。手写版本与 Clang 输出的栈帧布局接近，但奇偶判断采用了不同指令。

优化实验显示，`-O2` 会改变变量表示，并化简分支和算术；最终是否向量化还取决于目标架构。本例在 x86-64 上出现向量运算，在不含 RVV 的 RV64 目标上仍是标量循环。

AscendNPU IR 实验展示了硬件约束如何进入较低层表示。PlanMemory 用例将四个临时分配转换为偏移为 0 的 `pointer_cast`，计算操作保持不变；InjectSync 则在数据搬运与向量计算之间加入 flag/wait，并在函数末尾加入 barrier。三条上游 RUN 命令及其 FileCheck 断言均通过。Arith/Tensor 到 HFusion 的部分仍是资料整理，实测范围从 HIVM 开始。

整体上，本次实验完成了处理流程观察、手写实现和固定用例验证，并通过真实 Pass 输出补充 Ascend IR 实验。

参考文献

[1] LLVM Project. *MLIR Language Reference*. https://mlir.llvm.org/docs/LangRef/.

[2] LLVM Project. *MLIR Rationale*. https://github.com/llvm/llvm-project/blob/main/mlir/docs/Rationale/Rationale.md.

[3] LLVM Project. *Targeting LLVM IR from MLIR*. https://mlir.llvm.org/docs/TargetLLVMIR/.

[4] Ascend. *AscendNPU-IR Framework Integration*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/developer_guide/conversion/framework_interface.md.

[5] Ascend. *HIVM VecAdd add.mlir*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/add.mlir.

[6] Ascend. *HIVM VecAdd README and host wrapper*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/README.md.

[7] Ascend. *Plan Memory*. https://ascendnpu-ir.gitcode.com/en/developer_guide/features/PlanMemory/PlanMemory.html.

[8] Ascend. *HIVM Passes*. https://ascendnpu-ir.gitcode.com/en/developer_guide/passes/hivm_passes.html.

[9] Ascend. *PlanMemory Test Case*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Dialect/HIVM/plan-memory.mlir.

[10] Ascend. *InjectSync Test Case*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Dialect/HIVM/inject-sync.mlir.

[11] Ascend. *AscendNPU-IR Build and Installation*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/introduction/quick_start/installing_guide.md.

A 实验复现说明与材料索引

A.1 环境、文件组织与外部依赖

原实验以 `scripts/` 组织命令，以 `logs/` 和 `records/` 保存输出与过程记录。自动生成文件保存在 `generated/`，手写 LLVM IR 与汇编分别保存在 `llvm/` 和 `riscv/`，便于区分工具输出与人工实现。

这些名称只用于识别实验材料，阅读和核查无需访问原仓库。附录 B 至 E 已收录必要文本。

如需复现，可在 WSL 工作目录中按下列布局将附录代码保存为对应文件；`scripts/` 中的历史脚本名仅说明原执行阶段，直接复现可使用附录中的展开命令。外部工具链、课程提供的 `libsysy_riscv.a` 以及固定提交的 AscendNPU-IR/LLVM 源码仍需另行取得；报告不包含这些二进制依赖。`host_runtime.c` 是基于标准 C I/O 的宿主机适配库，不是课程 RISC-V 运行库。

Listing 1: 附录代码的建议保存布局

```
source/example.sy
source/example_clang.c
source/host_runtime.c
llvm/manual.ll
riscv/manual.s
mlir/vecadd_hivm.mlir
mlir/plan_memory_before.mlir
mlir/sync_before.mlir
generated/                  # generated artifacts
exploration/                # O0/O2 artifacts
```

A.2 各阶段复现脚本总表

下表保留原实验的阶段脚本及用途。本次报告重组使用 WSL 中已有的 XeLaTeX/latexmk 编译；表内 Tectonic 是历史构建入口。报告源稿已直接嵌入附录文本，不需要原实验目录中的代码文件参与排版。编译命令为 `latexmk -xelatex -interaction=nonstopmode -halt-on-error report.tex`。

表 10: 各阶段复现入口

脚本	作用
`00_environment_check.sh`	记录系统与工具版本，作为环境快照
`01_reference.sh`	编译宿主机 Ground Truth 并跑六组测试
`02_pipeline.sh`	生成 `.i / .ll / .s / .o` 并显式链接
`03_llvm_analysis.sh`、`03_llvm_validation.sh`	抽取 IR 做 CFG 分析；验证手写 `manual.ll`
`04_riscv_reference.sh`、`05_riscv_validation.sh`	生成 RV64 参考汇编；汇编链接手写 `manual.s` 并在 QEMU 运行
`06_optimization.sh`	生成 O0/O2 三种目标的对照与 diff
`09_final_validation.sh`	三路实现统一比对并输出总表
`10_build_report.sh`	用 Tectonic 编译本报告
`11_ascend_configure.sh`	配置构建目录并确认 `bishengir-opt` target
`12_ascend_build.sh`	构建 `bishengir-opt`
`13_ascend_pass_runs.sh`	运行三个上游 Pass 测试并保存 Before/After

A.3 正文结论与附录材料对应表

表 11: 正文与独立过程材料的对应关系

正文主题	材料位置
输入程序与运行库	附录 B.1、B.2
LLVM/RISC-V 手写实现	附录 B.3、B.4
处理流程、ELF 与生成结果	附录 C.1–C.3
链接、执行与六组结果	附录 C.4、C.5
优化计数、内联与目标差异	附录 D
Ascend 构建与上游测试	附录 E.2、E.3
PlanMemory/InjectSync 变化	附录 E.4、E.5

B 基础实验输入与完整手写实现

B.1 SysY 源程序与 Clang 输入版本

Listing 2: 完整 SysY 源程序

```
int calc(int n) {
    int i = 1;
    int sum = 0;

    while (i <= n) {
        if (i % 2 == 0) {
            sum = sum + i * 2;
        } else {
            sum = sum + i;
        }

        i = i + 1;
    }

    return sum;
}

int main() {
    int n;
    n = getint();

    int result;
    result = calc(n);

    putint(result);

    return 0;
}
```

Listing 3: 补充 I/O 声明后的 Clang 输入

```
int getint(void);
void putint(int x);

int calc(int n) {
    int i = 1;
    int sum = 0;

    while (i <= n) {
        if (i % 2 == 0) {
            sum = sum + i * 2;
        } else {
            sum = sum + i;
        }
```

```
14
15          i = i + 1;
16      }
17
18      return sum;
19  }
20
21  int main(void) {
22      int n;
23      n = getint();
24
25      int result;
26      result = calc(n);
27
28      putint(result);
29
30      return 0;
31  }
```

B.2 宿主机输入输出运行库

Listing 4: 宿主机 I/O 适配库

```
1  #include <stdio.h>
2
3  int getint(void) {
4      int x;
5      scanf("%d", &x);
6      return x;
7  }
8
9  void putint(int x) {
10     printf("%d\n", x);
11 }
```

B.3 完整手写 LLVM IR

Listing 5: 完整手写 LLVM 模块

```
1  ; Hand-written LLVM IR for source/example.sy.
2  ; The control-flow graph is expressed with named basic blocks and memory slots.
3
4  source_filename = "manual.ll"
5  target triple = "x86_64-pc-linux-gnu"
6
7  declare i32 @getint()
8  declare void @putint(i32)
9
```

```
define i32 @calc(i32 %n) {
entry:
  %i.addr = alloca i32, align 4
  %sum.addr = alloca i32, align 4
  store i32 1, i32* %i.addr, align 4
  store i32 0, i32* %sum.addr, align 4
  br label %while.cond

while.cond:
  %i.cond = load i32, i32* %i.addr, align 4
  %continue = icmp sle i32 %i.cond, %n
  br i1 %continue, label %if.cond, label %while.end

if.cond:
  %i.parity = load i32, i32* %i.addr, align 4
  %remainder = srem i32 %i.parity, 2
  %is.even = icmp eq i32 %remainder, 0
  br i1 %is.even, label %if.then, label %if.else

if.then:
  %sum.even = load i32, i32* %sum.addr, align 4
  %i.even = load i32, i32* %i.addr, align 4
  %twice.i = mul nsw i32 %i.even, 2
  %sum.even.next = add nsw i32 %sum.even, %twice.i
  store i32 %sum.even.next, i32* %sum.addr, align 4
  br label %if.end

if.else:
  %sum.odd = load i32, i32* %sum.addr, align 4
  %i.odd = load i32, i32* %i.addr, align 4
  %sum.odd.next = add nsw i32 %sum.odd, %i.odd
  store i32 %sum.odd.next, i32* %sum.addr, align 4
  br label %if.end

if.end:
  %i.current = load i32, i32* %i.addr, align 4
  %i.next = add nsw i32 %i.current, 1
  store i32 %i.next, i32* %i.addr, align 4
  br label %while.cond

while.end:
  %sum.result = load i32, i32* %sum.addr, align 4
  ret i32 %sum.result
}

define i32 @main() {
entry:
  %n = call i32 @getint()
  %result = call i32 @calc(i32 %n)
```

```
  call void @putint(i32 %result)
  ret i32 0
}
```

B.4 完整手写 RISC-V 汇编

Listing 6: 完整手写 RV64 汇编

```
    .section .text
    .align 2

    .globl calc
    .type calc, @function
calc:
    addi sp, sp, -32
    sd ra, 24(sp)
    sd s0, 16(sp)
    addi s0, sp, 32
    sw a0, -20(s0)       # n
    li t0, 1
    sw t0, -24(s0)       # i = 1
    sw zero, -28(s0)     # sum = 0

.Lwhile_cond:
    lw t0, -24(s0)       # i
    lw t1, -20(s0)       # n
    blt t1, t0, .Lwhile_end

.Lif_cond:
    li t1, 2
    remw t2, t0, t1
    beqz t2, .Lif_then
    j .Lif_else

.Lif_then:
    lw t0, -28(s0)       # sum
    lw t1, -24(s0)       # i
    slliw t1, t1, 1      # 2 * i
    addw t0, t0, t1
    sw t0, -28(s0)
    j .Lif_end

.Lif_else:
    lw t0, -28(s0)       # sum
    lw t1, -24(s0)       # i
    addw t0, t0, t1
    sw t0, -28(s0)

.Lif_end:
```

```
    lw t0, -24(s0)
    addiw t0, t0, 1
    sw t0, -24(s0)
    j .Lwhile_cond

.Lwhile_end:
    lw a0, -28(s0)       # integer return value
    ld s0, 16(sp)
    ld ra, 24(sp)
    addi sp, sp, 32
    ret
    .size calc, .-calc

    .align 2
    .globl main
    .type main, @function
main:
    addi sp, sp, -16
    sd ra, 8(sp)
    call getint          # input/return value in a0
    call calc            # argument and return value in a0
    call putint          # output argument in a0
    li a0, 0
    ld ra, 8(sp)
    addi sp, sp, 16
    ret
    .size main, .-main

    .section .note.GNU-stack,"",@progbits
```

C 编译流程与运行验证记录

C.1 预处理、IR、汇编和目标文件生成命令

以下命令在附录 A 所述工作目录执行，工具版本见正文。命令展开了历史脚本的核心步骤，不依赖原作者的环境初始化文件。

Listing 7: 基础流程展开命令

```
mkdir -p generated exploration
clang -E source/example_clang.c -o generated/example.i
clang -S -emit-llvm -O0 source/example_clang.c -o generated/example.ll
clang -S -O0 source/example_clang.c -o generated/example_x86.s
clang -c -O0 source/example_clang.c -o generated/example.o
clang -c -O0 source/host_runtime.c -o generated/host_runtime.o
file generated/example.o generated/host_runtime.o
readelf -h -S -s -r generated/example.o
readelf -s -r generated/host_runtime.o
clang -v generated/example.o generated/host_runtime.o -o generated/reference_linked
clang --target=riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
  -S -O0 source/example_clang.c -o generated/example_riscv.s
```

C.2 ELF 文件头、符号、重定位与链接日志摘录

以下摘录来自既有实验日志，保留文件头、全部节区和两份目标文件的符号与重定位检查，以及 Clang 的实际链接器调用；未重新执行该实验。

Listing 8: ELF 检查与显式链接的原始记录

```
[ELF object inspection]
generated/example.o:      ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
generated/host_runtime.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          776 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
  Size of section headers:           64 (bytes)
```

```
  Number of section headers:         10
  Section header string table index: 1
There are 10 section headers, starting at offset 0x308:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .strtab           STRTAB           0000000000000000  0000028b
       000000000000007a  0000000000000000           0     0     1
  [ 2] .text             PROGBITS         0000000000000000  00000040
       00000000000000a2  0000000000000000  AX       0     0     16
  [ 3] .rela.text        RELA             0000000000000000  00000210
       0000000000000048  0000000000000018   I       9     2     8
  [ 4] .comment          PROGBITS         0000000000000000  000000e2
       0000000000000028  0000000000000001  MS       0     0     1
  [ 5] .note.GNU-stack   PROGBITS         0000000000000000  0000010a
       0000000000000000  0000000000000000           0     0     1
  [ 6] .eh_frame         X86_64_UNWIND    0000000000000000  00000110
       0000000000000058  0000000000000000   A       0     0     8
  [ 7] .rela.eh_frame    RELA             0000000000000000  00000258
       0000000000000030  0000000000000018   I       9     6     8
  [ 8] .llvm_addrsig     LOOS+0xfff4c03   0000000000000000  00000288
       0000000000000003  0000000000000000   E       9     0     1
  [ 9] .symtab           SYMTAB           0000000000000000  00000168
       00000000000000a8  0000000000000018           1     3     8
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  D (mbind), l (large), p (processor specific)

Symbol table '.symtab' contains 7 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS example_clang.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    2 .text
     3: 0000000000000000   100 FUNC    GLOBAL DEFAULT    2 calc
     4: 0000000000000070    50 FUNC    GLOBAL DEFAULT    2 main
     5: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND getint
     6: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND putint

Relocation section '.rela.text' at offset 0x210 contains 3 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000080  000500000004 R_X86_64_PLT32    0000000000000000 getint - 4
00000000008b  000300000004 R_X86_64_PLT32    0000000000000000 calc - 4
000000000096  000600000004 R_X86_64_PLT32    0000000000000000 putint - 4

```

```
Relocation section '.rela.eh_frame' at offset 0x258 contains 2 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000020  000200000002 R_X86_64_PC32     0000000000000000 .text + 0
000000000040  000200000002 R_X86_64_PC32     0000000000000000 .text + 70

Symbol table '.symtab' contains 9 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS host_runtime.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    2 .text
     3: 0000000000000000     3 OBJECT  LOCAL  DEFAULT    4 .L.str
     4: 0000000000000003     4 OBJECT  LOCAL  DEFAULT    4 .L.str.1
     5: 0000000000000000    35 FUNC    GLOBAL DEFAULT    2 getint
     6: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND __isoc99_scanf
     7: 0000000000000030    34 FUNC    GLOBAL DEFAULT    2 putint
     8: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND printf

Relocation section '.rela.text' at offset 0x1f8 contains 4 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
00000000000b  000300000002 R_X86_64_PC32     0000000000000000 .L.str - 4
000000000016  000600000004 R_X86_64_PLT32    0000000000000000 __isoc99_scanf - 4
000000000041  000400000002 R_X86_64_PC32     0000000000000003 .L.str.1 - 4
000000000048  000800000004 R_X86_64_PLT32    0000000000000000 printf - 4

Relocation section '.rela.eh_frame' at offset 0x258 contains 2 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000020  000200000002 R_X86_64_PC32     0000000000000000 .text + 0
000000000040  000200000002 R_X86_64_PC32     0000000000000000 .text + 30

[Explicit link]
+ clang -v generated/example.o generated/host_runtime.o -o generated/reference_linked
Ubuntu clang version 14.0.0-1ubuntu1.1
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/bin
Found candidate GCC installation: /usr/bin/../lib/gcc/x86_64-linux-gnu/11
Selected GCC installation: /usr/bin/../lib/gcc/x86_64-linux-gnu/11
Candidate multilib: .;@m64
Selected multilib: .;@m64
 "/usr/bin/ld" -pie -z relro --hash-style=gnu --build-id --eh-frame-hdr -m elf_x86_64 -
    dynamic-linker /lib64/ld-linux-x86-64.so.2 -o generated/reference_linked /lib/x86_64-
    linux-gnu/Scrt1.o /lib/x86_64-linux-gnu/crti.o /usr/bin/../lib/gcc/x86_64-linux-gnu/11/
    crtbeginS.o -L/usr/bin/../lib/gcc/x86_64-linux-gnu/11 -L/usr/bin/../lib/gcc/x86_64-
    linux-gnu/11/../../../../lib64 -L/lib/x86_64-linux-gnu -L/lib/../lib64 -L/usr/lib/
    x86_64-linux-gnu -L/usr/lib/../lib64 -L/usr/lib/llvm-14/bin/../lib -L/lib -L/usr/lib
    generated/example.o generated/host_runtime.o -lgcc --as-needed -lgcc_s --no-as-needed -
    lc -lgcc --as-needed -lgcc_s --no-as-needed /usr/bin/../lib/gcc/x86_64-linux-gnu/11/
    crtendS.o /lib/x86_64-linux-gnu/crtn.o
```

```
generated/reference_linked: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
    dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
    ee167d0e6a32f819b8e9e38888a88efc3c12effb, for GNU/Linux 3.2.0, not stripped
```

C.3 自动生成的 LLVM IR 与 RV64 参考汇编

以下两份基础产物完整收录，可与附录 B 的手写实现逐条对照。

Listing 9: Clang -O0 生成的完整 LLVM 模块

```
; ModuleID = 'source/example_clang.c'
source_filename = "source/example_clang.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @calc(i32 noundef %0) #0 {
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 %0, i32* %2, align 4
  store i32 1, i32* %3, align 4
  store i32 0, i32* %4, align 4
  br label %5

5:                                                ; preds = %22, %1
  %6 = load i32, i32* %3, align 4
  %7 = load i32, i32* %2, align 4
  %8 = icmp sle i32 %6, %7
  br i1 %8, label %9, label %25

9:                                                ; preds = %5
  %10 = load i32, i32* %3, align 4
  %11 = srem i32 %10, 2
  %12 = icmp eq i32 %11, 0
  br i1 %12, label %13, label %18

13:                                               ; preds = %9
  %14 = load i32, i32* %4, align 4
  %15 = load i32, i32* %3, align 4
  %16 = mul nsw i32 %15, 2
  %17 = add nsw i32 %14, %16
  store i32 %17, i32* %4, align 4
  br label %22

18:                                               ; preds = %9
  %19 = load i32, i32* %4, align 4
  %20 = load i32, i32* %3, align 4
  %21 = add nsw i32 %19, %20
```

```
  store i32 %21, i32* %4, align 4
  br label %22

22:                                               ; preds = %18, %13
  %23 = load i32, i32* %3, align 4
  %24 = add nsw i32 %23, 1
  store i32 %24, i32* %3, align 4
  br label %5, !llvm.loop !6

25:                                               ; preds = %5
  %26 = load i32, i32* %4, align 4
  ret i32 %26
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  store i32 0, i32* %1, align 4
  %4 = call i32 @getint()
  store i32 %4, i32* %2, align 4
  %5 = load i32, i32* %2, align 4
  %6 = call i32 @calc(i32 noundef %5)
  store i32 %6, i32* %3, align 4
  %7 = load i32, i32* %3, align 4
  call void @putint(i32 noundef %7)
  ret i32 0
}

declare i32 @getint() #1

declare void @putint(i32 noundef) #1

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-
    width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="
    x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-
    size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "
    tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 7, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 1}
!4 = !{i32 7, !"frame-pointer", i32 2}
```

```
!5 = !{!"Ubuntu clang version 14.0.0-1ubuntu1.1"}
!6 = distinct !{!6, !7}
!7 = !{!"llvm.loop.mustprogress"}
```

Listing 10: Clang -O0 生成的完整 RV64 参考汇编

```
        .text
        .attribute      4, 16
        .attribute      5, "rv64i2p0_m2p0_a2p0_f2p0_d2p0_c2p0"
        .file   "example_clang.c"
        .globl  calc
        .p2align        1
        .type   calc,@function
calc:
        addi    sp, sp, -32
        sd      ra, 24(sp)
        sd      s0, 16(sp)
        addi    s0, sp, 32
        sw      a0, -20(s0)
        li      a0, 1
        sw      a0, -24(s0)
        li      a0, 0
        sw      a0, -28(s0)
        j       .LBB0_1
.LBB0_1:
        lw      a1, -24(s0)
        lw      a0, -20(s0)
        blt     a0, a1, .LBB0_6
        j       .LBB0_2
.LBB0_2:
        lwu     a0, -24(s0)
        srli    a1, a0, 31
        add     a1, a1, a0
        andi    a1, a1, -2
        subw    a0, a0, a1
        li      a1, 0
        bne     a0, a1, .LBB0_4
        j       .LBB0_3
.LBB0_3:
        lw      a0, -28(s0)
        lw      a1, -24(s0)
        slliw   a1, a1, 1
        addw    a0, a0, a1
        sw      a0, -28(s0)
        j       .LBB0_5
.LBB0_4:
        lw      a0, -28(s0)
        lw      a1, -24(s0)
        addw    a0, a0, a1
        sw      a0, -28(s0)
```

```
        j       .LBB0_5
.LBB0_5:
        lw      a0, -24(s0)
        addiw   a0, a0, 1
        sw      a0, -24(s0)
        j       .LBB0_1
.LBB0_6:
        lw      a0, -28(s0)
        ld      ra, 24(sp)
        ld      s0, 16(sp)
        addi    sp, sp, 32
        ret
.Lfunc_end0:
        .size   calc, .Lfunc_end0-calc

        .globl  main
        .p2align        1
        .type   main,@function
main:
        addi    sp, sp, -48
        sd      ra, 40(sp)
        sd      s0, 32(sp)
        addi    s0, sp, 48
        li      a0, 0
        sd      a0, -40(s0)
        sw      a0, -20(s0)
        call    getint
        sw      a0, -24(s0)
        lw      a0, -24(s0)
        call    calc
        sw      a0, -28(s0)
        lw      a0, -28(s0)
        call    putint
        ld      a0, -40(s0)
        ld      ra, 40(sp)
        ld      s0, 32(sp)
        addi    sp, sp, 48
        ret
.Lfunc_end1:
        .size   main, .Lfunc_end1-main

        .ident  "Ubuntu clang version 14.0.0-1ubuntu1.1"
        .section        ".note.GNU-stack","",@progbits
        .addrsig
        .addrsig_sym calc
        .addrsig_sym getint
        .addrsig_sym putint
```

C.4 手写实现的编译、链接与执行命令

将 RISCV_GCC 指向已安装的交叉编译器，将 SYSY_LIB_DIR 指向课程运行库目录；这两个变量是外部依赖的位置，不是报告附带文件。

Listing 11: 手写实现的展开命令

```
clang -O0 llvm/manual.ll source/host_runtime.c -o generated/manual_llvm
export RISCV_GCC=riscv64-unknown-elf-gcc
export SYSY_LIB_DIR=/path/to/course/runtime/lib
"$RISCV_GCC" -march=rv64gc -mabi=lp64d -mcmodel=medany \
  -c riscv/manual.s -o generated/manual_riscv.o
"$RISCV_GCC" generated/manual_riscv.o -o generated/manual_riscv \
  -L"$SYSY_LIB_DIR" -lsysy_riscv -static -march=rv64gc -mabi=lp64d \
  -mcmodel=medany -Wl,--no-relax,-Ttext=0x90000000
printf "10\n" | generated/manual_llvm
printf "10\n" | qemu-riscv64 generated/manual_riscv
```

C.5 固定测试方法、逐组输出与一致性结果

测试只比较标准输出。以下循环按相同输入核对三种实现，QEMU 的 stderr 计时摘要不计入输出值。

Listing 12: 不依赖历史测试脚本的固定测试方法

```
while read -r input expected; do
  for mode in reference llvm riscv; do
    case "$mode" in
      reference) actual=$(printf "%s\n" "$input" | generated/reference_linked) ;;
      llvm) actual=$(printf "%s\n" "$input" | generated/manual_llvm) ;;
      riscv) actual=$(printf "%s\n" "$input" | qemu-riscv64 generated/manual_riscv) ;;
    esac
    test "$actual" = "$expected" || exit 1
    printf "%s n=%s -> %s PASS\n" "$mode" "$input" "$actual"
  done
done <<'CASES'
1 1
2 5
3 8
4 16
5 21
10 85
CASES
```

Listing 13: 原参考实现的六组记录

```
n=1 -> 1 PASS
n=2 -> 5 PASS
n=3 -> 8 PASS
n=4 -> 16 PASS
```

```
n=5 -> 21 PASS
n=10 -> 85 PASS
```

Listing 14: 显式链接参考实现的六组记录

```
n=1 -> 1 PASS
n=2 -> 5 PASS
n=3 -> 8 PASS
n=4 -> 16 PASS
n=5 -> 21 PASS
n=10 -> 85 PASS
```

Listing 15: 手写 LLVM 实现的六组记录

```
n=1 -> 1 PASS
n=2 -> 5 PASS
n=3 -> 8 PASS
n=4 -> 16 PASS
n=5 -> 21 PASS
n=10 -> 85 PASS
```

Listing 16: 手写 RISC-V 实现的六组记录

```
n=1 -> 1 PASS
n=2 -> 5 PASS
n=3 -> 8 PASS
n=4 -> 16 PASS
n=5 -> 21 PASS
n=10 -> 85 PASS
```

Listing 17: 三路统一比对的原始结果矩阵

```
| Input | Reference | LLVM | RISC-V | Result |
| ---: | ---: | ---: | ---: | :---: |
| 1 | 1 | 1 | 1 | PASS |
| 2 | 5 | 5 | 5 | PASS |
| 3 | 8 | 8 | 8 | PASS |
| 4 | 16 | 16 | 16 | PASS |
| 5 | 21 | 21 | 21 | PASS |
| 10 | 85 | 85 | 85 | PASS |
```

D 优化对照过程材料

D.1 O0/O2 生成命令与指令统计口径

Listing 18: 优化级别对照的生成命令

```
for level in 0 2; do
  clang -S -emit-llvm -O"$level" source/example_clang.c -o exploration/O"$level".ll
  clang -S -O"$level" source/example_clang.c -o exploration/O"$level".s
  clang --target=riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
    -S -O"$level" source/example_clang.c -o exploration/O"$level"_riscv.s
done
```

统计完整 LLVM 模块中非注释指令行的 `alloca/load/store/phi`；汇编长度统计完整文本行数，包含指令、标签、注释和伪指令，因此不能替代机器码大小或性能指标。

Listing 19: 依据现有产物复核的统计

```
exploration/O0.ll: alloca=6, load=11, store=9, phi=0
exploration/O2.ll: alloca=0, load=0, store=0, phi=32
exploration/O0.s: 87 lines
exploration/O2.s: 385 lines
exploration/O0_riscv.s: 91 lines
exploration/O2_riscv.s: 56 lines
```

D.2 LLVM IR 关键变化及原始片段

O0 的完整模块见附录 C.3。以下为 O2 原始模块的连续摘录；每段标明原行号，片段之间有省略，SSA 定义可位于未摘录的区域，不能把片段当作独立可编译模块。选择循环 phi、向量算术与归约、标量尾部以及 main 的入口以展示内联。

Listing 20: O2 LLVM IR 原第 27–44 行

```
  br label %15

15:                                               ; preds = %15, %13
  %16 = phi <4 x i32> [ zeroinitializer, %13 ], [ %37, %15 ]
  %17 = phi <4 x i32> [ zeroinitializer, %13 ], [ %38, %15 ]
  %18 = phi <4 x i32> [ <i32 1, i32 2, i32 3, i32 4>, %13 ], [ %39, %15 ]
  %19 = phi i32 [ 0, %13 ], [ %40, %15 ]
  %20 = add <4 x i32> %18, <i32 4, i32 4, i32 4, i32 4>
  %21 = and <4 x i32> %18, <i32 1, i32 1, i32 1, i32 1>
  %22 = and <4 x i32> %18, <i32 1, i32 1, i32 1, i32 1>
  %23 = xor <4 x i32> %21, <i32 1, i32 1, i32 1, i32 1>
  %24 = xor <4 x i32> %22, <i32 1, i32 1, i32 1, i32 1>
  %25 = shl nuw nsw <4 x i32> %18, %23
  %26 = shl nuw nsw <4 x i32> %20, %24
  %27 = add <4 x i32> %25, %16
  %28 = add <4 x i32> %26, %17
  %29 = add <4 x i32> %18, <i32 8, i32 8, i32 8, i32 8>
```

```
18   %30 = add <4 x i32> %18, <i32 12, i32 12, i32 12, i32 12>
```

Listing 21: O2 LLVM IR 原第 32–49 行

```
1   %18 = phi <4 x i32> [ <i32 1, i32 2, i32 3, i32 4>, %13 ], [ %39, %15 ]
2   %19 = phi i32 [ 0, %13 ], [ %40, %15 ]
3   %20 = add <4 x i32> %18, <i32 4, i32 4, i32 4, i32 4>
4   %21 = and <4 x i32> %18, <i32 1, i32 1, i32 1, i32 1>
5   %22 = and <4 x i32> %18, <i32 1, i32 1, i32 1, i32 1>
6   %23 = xor <4 x i32> %21, <i32 1, i32 1, i32 1, i32 1>
7   %24 = xor <4 x i32> %22, <i32 1, i32 1, i32 1, i32 1>
8   %25 = shl nuw nsw <4 x i32> %18, %23
9   %26 = shl nuw nsw <4 x i32> %20, %24
10  %27 = add <4 x i32> %25, %16
11  %28 = add <4 x i32> %26, %17
12  %29 = add <4 x i32> %18, <i32 8, i32 8, i32 8, i32 8>
13  %30 = add <4 x i32> %18, <i32 12, i32 12, i32 12, i32 12>
14  %31 = and <4 x i32> %18, <i32 1, i32 1, i32 1, i32 1>
15  %32 = and <4 x i32> %18, <i32 1, i32 1, i32 1, i32 1>
16  %33 = xor <4 x i32> %31, <i32 1, i32 1, i32 1, i32 1>
17  %34 = xor <4 x i32> %32, <i32 1, i32 1, i32 1, i32 1>
18  %35 = shl nuw nsw <4 x i32> %29, %33
```

Listing 22: O2 LLVM IR 原第 80–97 行

```
1   %60 = phi <4 x i32> [ %43, %42 ], [ %57, %49 ]
2   %61 = phi <4 x i32> [ %44, %42 ], [ %58, %49 ]
3   %62 = add <4 x i32> %61, %60
4   %63 = call i32 @llvm.vector.reduce.add.v4i32(<4 x i32> %62)
5   %64 = icmp eq i32 %6, %0
6   br i1 %64, label %77, label %65
7
8  65:                                               ; preds = %3, %59
9   %66 = phi i32 [ 0, %3 ], [ %63, %59 ]
10  %67 = phi i32 [ 1, %3 ], [ %7, %59 ]
11  br label %68
12
13 68:                                               ; preds = %65, %68
14  %69 = phi i32 [ %74, %68 ], [ %66, %65 ]
15  %70 = phi i32 [ %75, %68 ], [ %67, %65 ]
16  %71 = and i32 %70, 1
17  %72 = xor i32 %71, 1
18  %73 = shl nuw nsw i32 %70, %72
```

Listing 23: O2 LLVM IR 原第 106–123 行

```
1  }
2
3  ; Function Attrs: nounwind uwtable
4  define dso_local i32 @main() local_unnamed_addr #1 {
```

```
  %1 = tail call i32 @getint() #4
  %2 = icmp slt i32 %1, 1
  br i1 %2, label %77, label %3

3:                                                ; preds = %0
  %4 = icmp ult i32 %1, 8
  br i1 %4, label %65, label %5

5:                                                ; preds = %3
  %6 = and i32 %1, -8
  %7 = or i32 %6, 1
  %8 = add i32 %6, -8
  %9 = lshr exact i32 %8, 3
  %10 = add nuw nsw i32 %9, 1
```

O2 的 `main` 函数体中未出现 `call @calc`，但独立 `@calc` 定义仍保留；此结论来自对完整函数体的检查。

D.3 x86-64 与 RV64 汇编对照片段

Listing 24: O0.s 原第 2–29 行摘录

```
        .file   "example_clang.c"
        .globl  calc                            # -- Begin function calc
        .p2align        4, 0x90
        .type   calc,@function
calc:                                   # @calc
        .cfi_startproc
# %bb.0:
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset %rbp, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register %rbp
        movl    %edi, -4(%rbp)
        movl    $1, -8(%rbp)
        movl    $0, -12(%rbp)
.LBB0_1:                                # =>This Inner Loop Header: Depth=1
        movl    -8(%rbp), %eax
        cmpl    -4(%rbp), %eax
        jg      .LBB0_6
# %bb.2:                                #   in Loop: Header=BB0_1 Depth=1
        movl    -8(%rbp), %eax
        movl    $2, %ecx
        cltd
        idivl   %ecx
        cmpl    $0, %edx
        jne     .LBB0_4
# %bb.3:                                #   in Loop: Header=BB0_1 Depth=1
```

```
28          movl    -12(%rbp), %eax
```

Listing 25: O2.s 原第 79–112 行摘录

```
1           pxor    %xmm2, %xmm2
2           .p2align        4, 0x90
3   .LBB0_7:                                # =>This Inner Loop Header: Depth=1
4           movdqa  %xmm1, %xmm7
5           paddd   %xmm8, %xmm7
6           movdqa  %xmm1, %xmm4
7           pandn   %xmm9, %xmm4
8           pslld   $23, %xmm4
9           paddd   %xmm10, %xmm4
10          cvttps2dq       %xmm4, %xmm4
11          movdqa  %xmm1, %xmm5
12          pmuludq %xmm4, %xmm5
13          pshufd  $232, %xmm5, %xmm6              # xmm6 = xmm5[0,2,2,3]
14          pshufd  $245, %xmm1, %xmm3              # xmm3 = xmm1[1,1,3,3]
15          pshufd  $245, %xmm4, %xmm5              # xmm5 = xmm4[1,1,3,3]
16          pmuludq %xmm5, %xmm3
17          pshufd  $232, %xmm3, %xmm3              # xmm3 = xmm3[0,2,2,3]
18          punpckldq       %xmm3, %xmm6            # xmm6 = xmm6[0],xmm3[0],xmm6[1],xmm3[1]
19          paddd   %xmm0, %xmm6
20          pshufd  $245, %xmm7, %xmm0              # xmm0 = xmm7[1,1,3,3]
21          pmuludq %xmm4, %xmm7
22          pshufd  $232, %xmm7, %xmm7              # xmm7 = xmm7[0,2,2,3]
23          pmuludq %xmm5, %xmm0
24          pshufd  $232, %xmm0, %xmm0              # xmm0 = xmm0[0,2,2,3]
25          punpckldq       %xmm0, %xmm7            # xmm7 = xmm7[0],xmm0[0],xmm7[1],xmm0[1]
26          paddd   %xmm2, %xmm7
27          movdqa  %xmm1, %xmm0
28          paddd   %xmm11, %xmm0
29          movdqa  %xmm1, %xmm3
30          paddd   %xmm12, %xmm3
31          pshufd  $245, %xmm0, %xmm2              # xmm2 = xmm0[1,1,3,3]
32          pmuludq %xmm4, %xmm0
33          pshufd  $232, %xmm0, %xmm0              # xmm0 = xmm0[0,2,2,3]
34          pmuludq %xmm5, %xmm2
```

Listing 26: O2 riscv.s 原第 4–35 行摘录

```
1           .file   "example_clang.c"
2           .globl  calc
3           .p2align        1
4           .type   calc,@function
5   calc:
6           li      a1, 0
7           blez    a0, .LBB0_3
8           li      a2, 1
9   .LBB0_2:
```

```
        mv      a3, a2
        not     a2, a2
        andi    a2, a2, 1
        sllw    a2, a3, a2
        addw    a1, a1, a2
        addiw   a2, a3, 1
        blt     a3, a0, .LBB0_2
.LBB0_3:
        mv      a0, a1
        ret
.Lfunc_end0:
        .size   calc, .Lfunc_end0-calc

        .globl  main
        .p2align        1
        .type   main,@function
main:
        addi    sp, sp, -16
        sd      ra, 8(sp)
        call    getint
        li      a1, 0
        blez    a0, .LBB1_3
        li      a2, 1
```

RV64 的 O0 完整参考汇编见附录 C.3。O2 片段可见标量寄存器与位运算；该目标为 `rv64gc`，没有启用 RVV。x86-64 片段中的 XMM 与 `paddd` 对应向量运算。以上仅报告结构变化，不报告耗时、能耗或速度提升。

E AscendNPU IR 实验过程材料

E.1 官方 VecAdd HIVM 完整输入

Listing 27: 官方 VecAdd 的完整本地输入

```
module {
  func.func @add(%arg0: memref<16xi16, #hivm.address_space<gm>>, %arg1: memref<16xi16, #hivm
    .address_space<gm>>, %arg2: memref<16xi16, #hivm.address_space<gm>>) attributes {hacc.
    entry, hacc.function_kind = #hacc.function_kind<DEVICE>} {
    %alloc = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
    hivm.hir.load ins(%arg0 : memref<16xi16, #hivm.address_space<gm>>) outs(%alloc : memref
    <16xi16, #hivm.address_space<ub>>)
    %alloc_0 = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
    hivm.hir.load ins(%arg1 : memref<16xi16, #hivm.address_space<gm>>) outs(%alloc_0 :
    memref<16xi16, #hivm.address_space<ub>>)
    %alloc_1 = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
    hivm.hir.vadd ins(%alloc, %alloc_0 : memref<16xi16, #hivm.address_space<ub>>, memref<16
    xi16, #hivm.address_space<ub>>) outs(%alloc_1 : memref<16xi16, #hivm.address_space<ub
    >>)
    hivm.hir.store ins(%alloc_1 : memref<16xi16, #hivm.address_space<ub>>) outs(%arg2 :
    memref<16xi16, #hivm.address_space<gm>>)
    return
  }
}
```

E.2 构建版本、配置、故障处理与完整链接补丁

原实验使用 CMake 4.4.4、Ninja 1.13.2；AscendNPU-IR 提交为 `2ac1b376`，LLVM 子项目提交为 `06913395`，产出的工具为 bishengir-opt 2.0.0 / LLVM 19.1.7 Release。原实验采用六路编译，并记录约 4.8 GiB 内存峰值，未触发 3 GiB swap。

以下配置取自成功构建所用脚本。`REPO_DIR` 为固定版本上游源码的本地位置，`LLVM_SRC` 为其 `third-party/llvm-project` 子模块，`BUILD_DIR` 为构建目录，须按本地实际位置设置；采用用户级虚拟环境中的 CMake/Ninja。

Listing 28: 原构建配置及目标命令

```
export REPO_DIR=/path/to/ascendnpu-ir
export LLVM_SRC="$REPO_DIR/third-party/llvm-project"
export BUILD_DIR="$REPO_DIR/build"
cmake "$LLVM_SRC/llvm" -G Ninja -B "$BUILD_DIR" \
    -DCMAKE_C_COMPILER=clang \
    -DCMAKE_CXX_COMPILER=clang++ \
    -DCMAKE_BUILD_TYPE=Release \
    -DLLVM_ENABLE_PROJECTS=mlir \
    -DLLVM_EXTERNAL_PROJECTS=bishengir \
    -DLLVM_EXTERNAL_BISHENGIR_SOURCE_DIR="$REPO_DIR" \
    -DLLVM_TARGETS_TO_BUILD=host \
```

```
    -DLLVM_ENABLE_ASSERTIONS=OFF \
    -DLLVM_ENABLE_LLD=OFF \
    -DLLVM_USE_LINKER=gold \
    -DLLVM_PARALLEL_LINK_JOBS=1 \
    -DBUILD_SHARED_LIBS=OFF \
    -DBSPUB_DAVINCI_BISHENGIR=ON \
    -DLLVM_BSPUB_DAVINCI_BISHENGIR=ON \
    -DLLVM_BSPUB_DAVINCI_BISHENGIR_A5=ON \
    -DLLVM_BSPUB_DAVINCI_BISHENGIR_A5_NPUIR=ON \
    -DBISHENGIR_ENABLE_PM_CL_OPTIONS=ON \
    -DBISHENGIR_ENABLE_TRITON_COMPILE=ON \
    -DBISHENGIR_BUILD_HIVMC=ON \
    -DBISHENGIR_BUILD_TEMPLATE=OFF \
    -DSHMEM_BUILD_TEMPLATE=OFF \
    -DMLIR_ENABLE_BINDINGS_PYTHON=OFF
ninja -C "$BUILD_DIR" BiShengIRSkewShapeTable
ninja -C "$BUILD_DIR" bishengir-opt -j 6
ninja -C "$BUILD_DIR" FileCheck -j 6
```

构建经历四轮尝试：先遇到生成头文件 SkewShapeTable.inc 的依赖缺口，再遇到关闭 HIVMC 导致的未定义符号，最后遇到 HIVMCLib 与 A3/A5 的静态库循环。先单独构建 BiShengIRSkewShapeTable 生成头文件，并将 BISHENGIR_BUILD_HIVMC 恢复为上游默认的 ON，最终以以下链接组补丁解决循环。复现时应在配置与编译前，将下列补丁应用于固定版本源码；前面的配置清单已经采用成功配置。尝试添加 PUBLIC 依赖与 LINK_INTERFACE_MULTIPLICITY 时，CMake 因循环包含 OBJECT_LIBRARY 拒绝配置，该尝试已回退。补丁只调整该可执行文件的链接参数，不修改 Pass；它是本地处理方式，不是上游正式修复。以下为完整补丁。

Listing 29: 完整静态库链接组补丁

```
diff --git a/bishengir/tools/bishengir-opt/CMakeLists.txt b/bishengir/tools/bishengir-opt/
    CMakeLists.txt
index 8b61528..b188f13 100644
--- a/bishengir/tools/bishengir-opt/CMakeLists.txt
+++ b/bishengir/tools/bishengir-opt/CMakeLists.txt
@@ -43,4 +43,14 @@ endif()

 target_link_libraries(bishengir-opt PRIVATE ${LIBS})

+# BiShengIRHIVMCLib 与 BiShengIRHIVMCLibA3/A5 之间存在静态库循环:
+# A3/A5 引用 HIVMCLib 的 setSanitizerAddrArgName, 而 HIVMCLib 又调用
+# runHIVMCCompileA3/A5。静态库单遍扫描会漏解该循环, 导致
+# "undefined reference to setSanitizerAddrArgName"。
+# 这里用 LINK_GROUP:RESCAN 让链接器对这组库重复扫描
+# （在 GNU 工具链上生成 --start-group/--end-group）,
+# 不引入 CMake 层面的依赖环, 因此不会与 OBJECT_LIBRARY 冲突。
+target_link_libraries(bishengir-opt PRIVATE
+  "$<LINK_GROUP:RESCAN,BiShengIRHIVMCLib,BiShengIRHIVMCLibA3,BiShengIRHIVMCLibA5>")
+
 mlir_check_all_link_libraries(bishengir-opt)
```

E.3 三条上游 RUN/FileCheck 命令与通过记录

BIN、FC 指向上述构建的工具，TESTS 指向同一提交的上游测试目录。下列三条命令使用完整上游测试文件及其 CHECK 断言，不能只用后面摘出的单用例替代完整文件。

Listing 30: 三条上游测试命令

```
BIN="$BUILD_DIR/bin/bishengir-opt"
FC="$BUILD_DIR/bin/FileCheck"
TESTS="$REPO_DIR/bishengir/test/Dialect/HIVM"
printf '\n--- plan-memory.mlir 默认策略 ---\n'
"$BIN" "$TESTS/plan-memory.mlir" -hacc-append-device-spec=target=Ascend910B1 \
    -hivm-plan-memory -split-input-file -verify-diagnostics 2>&1 \
  | "$FC" "$TESTS/plan-memory.mlir" && printf 'RESULT: PASS\n'

printf '\n--- plan-memory.mlir largest-first 策略 (CHECK-REORDER) ---\n'
"$BIN" "$TESTS/plan-memory.mlir" -hacc-append-device-spec=target=Ascend910B1 \
    -hivm-plan-memory=plan-memory-strategy=largest-first -split-input-file \
    -verify-diagnostics 2>&1 \
  | "$FC" "$TESTS/plan-memory.mlir" --check-prefix=CHECK-REORDER && printf 'RESULT: PASS\n'

printf '\n--- inject-sync.mlir ---\n'
"$BIN" -hivm-inject-sync -hivm-lower-multi-buffer-counter -split-input-file \
    "$TESTS/inject-sync.mlir" 2>&1 \
  | "$FC" "$TESTS/inject-sync.mlir" && printf 'RESULT: PASS\n'
```

Listing 31: 原始上游验证与单用例统计记录

```
bishengir-opt: bishengir-opt 2.0.0 (https://gitcode.com/Ascend/ascendnpu-ir.git 2ac1b376da1d
     2026-10-09)
FileCheck:     /home/lenovo/projects/Compiler-Labs/third_party/ascendnpu-ir/build/bin/
    FileCheck

=== [1] 上游 RUN 命令逐字执行 (含 FileCheck 断言) ===

--- plan-memory.mlir 默认策略 ---
RESULT: PASS

--- plan-memory.mlir largest-first 策略 (CHECK-REORDER) ---
RESULT: PASS

--- inject-sync.mlir ---
RESULT: PASS

=== [2] 单用例 Before / After ===
plan-memory  before alloc=5  after alloc=0  pointer_cast=4
inject-sync  before(set_flag 代码行)=0  after set_flag=2  after wait_flag=2  after barrier=1

Stage 13 AscendNPU-IR pass runs: PASS
```

原日志中的 `before alloc=5` 含一条 `// CHECK-NOT: memref.alloc()` 注释，真实分配指令为 4 条。正文及本附录按非注释操作统计，因此采用 4→0；同步指令统计同样排除 CHECK 注释。历史日志原样保留，避免将记录与复核结果混为一谈。

E.4 PlanMemory 单用例完整 Before/After

该用例含四个 `16x16x16xf16` UB buffer。输入保留原始 RUN/CHECK 注释；After 为真实工具输出，其中目标规格由前置 `-hacc-append-device-spec=target=Ascend910B1` 写入。

Listing 32: PlanMemory 单用例输出命令

```
"$BIN" mlir/plan_memory_before.mlir \
  -hacc-append-device-spec=target=Ascend910B1 -hivm-plan-memory \
  -split-input-file -verify-diagnostics > mlir/plan_memory_after.mlir
```

Listing 33: PlanMemory 完整 Before

```
// RUN: bishengir-opt %s -hacc-append-device-spec=target=Ascend910B1 -hivm-plan-memory -split-input-file -verify-diagnostics | FileCheck %s
// RUN: bishengir-opt %s -hacc-append-device-spec=target=Ascend910B1 -hivm-plan-memory=plan-memory-strategy=largest-first -split-input-file -verify-diagnostics | FileCheck %s --check-prefix=CHECK-REORDER

module {
  // CHECK-LABEL: func.func @test_mem_allocate_basic
  func.func @test_mem_allocate_basic(%src : memref<16x16x16xf16, #hivm.address_space<gm>>,
                                     %dst3 : memref<16x16x16xf16, #hivm.address_space<gm>>) {
    // CHECK-NOT: memref.alloc()
    // CHECK: %[[CONST0:.*]] = arith.constant 0 : i64
    // CHECK: {{.*}} = hivm.hir.pointer_cast(%[[CONST0]])
    %copy_in_ub = memref.alloc() : memref<16x16x16xf16, #hivm.address_space<ub>>
    // CHECK: {{.*}} = hivm.hir.pointer_cast(%[[CONST0]])
    %dst1 = memref.alloc() : memref<16x16x16xf16, #hivm.address_space<ub>>
    // CHECK: {{.*}} = hivm.hir.pointer_cast(%[[CONST0]])
    %dst2 = memref.alloc() : memref<16x16x16xf16, #hivm.address_space<ub>>
    // CHECK: {{.*}} = hivm.hir.pointer_cast(%[[CONST0]])
    %copy_out_ub = memref.alloc() : memref<16x16x16xf16, #hivm.address_space<ub>>
    hivm.hir.load ins(%src : memref<16x16x16xf16, #hivm.address_space<gm>>)
                  outs(%copy_in_ub : memref<16x16x16xf16, #hivm.address_space<ub>>)
    hivm.hir.vadd ins(%copy_in_ub, %copy_in_ub : memref<16x16x16xf16, #hivm.address_space<ub>>,
                      memref<16x16x16xf16, #hivm.address_space<ub>>)
                  outs(%dst1 : memref<16x16x16xf16, #hivm.address_space<ub>>)
    hivm.hir.vadd ins(%dst1, %dst1 : memref<16x16x16xf16, #hivm.address_space<ub>>,
                      memref<16x16x16xf16, #hivm.address_space<ub>>)
                  outs(%dst2 : memref<16x16x16xf16, #hivm.address_space<ub>>)
    hivm.hir.vadd ins(%dst2, %dst2 : memref<16x16x16xf16, #hivm.address_space<ub>>,
                      memref<16x16x16xf16, #hivm.address_space<ub>>)
                  outs(%copy_out_ub : memref<16x16x16xf16, #hivm.address_space<ub>>)
    hivm.hir.store ins(%copy_out_ub : memref<16x16x16xf16,#hivm.address_space<ub>>)
```

```
30                    outs(%dst3: memref<16x16x16xf16,#hivm.address_space<gm>>)
31    return
32  }
33 }
```

Listing 34: PlanMemory 完整 After

```
1 module attributes {dlti.target_system_spec = #dlti.target_system_spec<"NPU" : #hacc.
    target_device_spec<#dlti.dl_entry<"AI_CORE_COUNT", 24 : i32>, #dlti.dl_entry<"
    CUBE_CORE_COUNT", 24 : i32>, #dlti.dl_entry<"VECTOR_CORE_COUNT", 48 : i32>, #dlti.
    dl_entry<"UB_SIZE", 1572864 : i32>, #dlti.dl_entry<"L1_SIZE", 4194304 : i32>, #dlti.
    dl_entry<"L0A_SIZE", 524288 : i32>, #dlti.dl_entry<"L0B_SIZE", 524288 : i32>, #dlti.
    dl_entry<"L0C_SIZE", 1048576 : i32>, #dlti.dl_entry<"UB_ALIGN_SIZE", 256 : i32>, #dlti.
    dl_entry<"L1_ALIGN_SIZE", 256 : i32>, #dlti.dl_entry<"L0C_ALIGN_SIZE", 4096 : i32>>>,
    hacc.hivmc_compatible_print = false, hacc.hivmc_version = #hacc.hivmc_version<"0.0.0">,
     hacc.target = #hacc.target<"Ascend910B1">} {
2   func.func @test_mem_allocate_basic(%arg0: memref<16x16x16xf16, #hivm.address_space<gm>>, %
    arg1: memref<16x16x16xf16, #hivm.address_space<gm>>) {
3     %c0_i64 = arith.constant 0 : i64
4     %0 = hivm.hir.pointer_cast(%c0_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
5     %1 = hivm.hir.pointer_cast(%c0_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
6     %2 = hivm.hir.pointer_cast(%c0_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
7     %3 = hivm.hir.pointer_cast(%c0_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
8     hivm.hir.load ins(%arg0 : memref<16x16x16xf16, #hivm.address_space<gm>>) outs(%0 :
    memref<16x16x16xf16, #hivm.address_space<ub>>)
9     hivm.hir.vadd ins(%0, %0 : memref<16x16x16xf16, #hivm.address_space<ub>>, memref<16
    x16x16xf16, #hivm.address_space<ub>>) outs(%1 : memref<16x16x16xf16, #hivm.
    address_space<ub>>)
10    hivm.hir.vadd ins(%1, %1 : memref<16x16x16xf16, #hivm.address_space<ub>>, memref<16
    x16x16xf16, #hivm.address_space<ub>>) outs(%2 : memref<16x16x16xf16, #hivm.
    address_space<ub>>)
11    hivm.hir.vadd ins(%2, %2 : memref<16x16x16xf16, #hivm.address_space<ub>>, memref<16
    x16x16xf16, #hivm.address_space<ub>>) outs(%3 : memref<16x16x16xf16, #hivm.
    address_space<ub>>)
12    hivm.hir.store ins(%3 : memref<16x16x16xf16, #hivm.address_space<ub>>) outs(%arg1 :
    memref<16x16x16xf16, #hivm.address_space<gm>>)
13    return
14  }
15 }
```

E.5 InjectSync 单用例完整 Before/After

原始符号名按材料保留。该用例与 VecAdd 和 PlanMemory 用例不同；输入的 CHECK 注释是断言，不是已经存在的同步操作。

Listing 35: InjectSync 单用例输出命令

```
1 "$BIN" mlir/sync_before.mlir -hivm-inject-sync \
2   -hivm-lower-multi-buffer-counter -split-input-file > mlir/sync_after.mlir
```

Listing 36: InjectSync 完整 Before

```
module {
  func.func @test_mem_injcet_sync_basic(%arg0: memref<16x16x16xf16, #hivm.address_space<gm>>,
                                        %arg3: memref<16x16x16xf16, #hivm.address_space<gm>>)
  attributes {hacc.entry, hacc.function_kind = #hacc.function_kind<DEVICE>} {
    %c16384_i64 = arith.constant 16384 : i64
    %c8192_i64 = arith.constant 8192 : i64
    %c0_i64 = arith.constant 0 : i64
    %0 = hivm.hir.pointer_cast(%c0_i64) [] : memref<16x16x16xf16, #hivm.address_space<ub>>
    %2 = hivm.hir.pointer_cast(%c0_i64) [] : memref<16x16x16xf16, #hivm.address_space<ub>>
    %4 = hivm.hir.pointer_cast(%c8192_i64) [] : memref<16x16x16xf16, #hivm.address_space<ub>>
    %5 = hivm.hir.pointer_cast(%c16384_i64) [] : memref<16x16x16xf16, #hivm.address_space<ub>>
    hivm.hir.load ins(%arg0 : memref<16x16x16xf16, #hivm.address_space<gm>>)
                  outs(%0 : memref<16x16x16xf16, #hivm.address_space<ub>>)
    // CHECK: hivm.hir.set_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
    // CHECK: hivm.hir.wait_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
    hivm.hir.vadd ins(%0, %4 : memref<16x16x16xf16, #hivm.address_space<ub>>,
                  memref<16x16x16xf16, #hivm.address_space<ub>>)
                  outs(%0 : memref<16x16x16xf16, #hivm.address_space<ub>>)
    // CHECK: hivm.hir.set_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
    // CHECK: hivm.hir.wait_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
    hivm.hir.store ins(%0 : memref<16x16x16xf16, #hivm.address_space<ub>>)
                   outs(%arg3 : memref<16x16x16xf16, #hivm.address_space<gm>>)
    // CHECK: hivm.hir.pipe_barrier[<PIPE_ALL>]
    return
  }
}
```

Listing 37: InjectSync 完整 After

```
module {
  func.func @test_mem_injcet_sync_basic(%arg0: memref<16x16x16xf16, #hivm.address_space<gm>>, %arg1: memref<16x16x16xf16, #hivm.address_space<gm>>) attributes {hacc.entry, hacc.function_kind = #hacc.function_kind<DEVICE>} {
    %c16384_i64 = arith.constant 16384 : i64
    %c8192_i64 = arith.constant 8192 : i64
    %c0_i64 = arith.constant 0 : i64
    %0 = hivm.hir.pointer_cast(%c0_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
    %1 = hivm.hir.pointer_cast(%c0_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
    %2 = hivm.hir.pointer_cast(%c8192_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
    %3 = hivm.hir.pointer_cast(%c16384_i64) : memref<16x16x16xf16, #hivm.address_space<ub>>
    hivm.hir.load ins(%arg0 : memref<16x16x16xf16, #hivm.address_space<gm>>) outs(%0 : memref<16x16x16xf16, #hivm.address_space<ub>>)
    hivm.hir.set_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
    hivm.hir.wait_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
```

```
13      hivm.hir.vadd ins(%0, %2 : memref<16x16x16xf16, #hivm.address_space<ub>>, memref<16
        x16x16xf16, #hivm.address_space<ub>>) outs(%0 : memref<16x16x16xf16, #hivm.
        address_space<ub>>)
14      hivm.hir.set_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
15      hivm.hir.wait_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
16      hivm.hir.store ins(%0 : memref<16x16x16xf16, #hivm.address_space<ub>>) outs(%arg1 :
        memref<16x16x16xf16, #hivm.address_space<gm>>)
17      hivm.hir.pipe_barrier[<PIPE_ALL>]
18      return
19    }
20  }
```